Datenformat der Messdaten (Test-PCB → PC)

Header, Rohdaten, Prüfsumme · Vorschlag Firmware-Funktion und Python-Empfang

Stand 08.11.2026 · Vorschlag, noch nicht in der Firmware umgesetzt

1 Zweck

Diese Doku legt fest, **wie die Firmware Messdaten an den PC schickt**. Firmware (`measure.c`) und Python-Programm müssen sich beide daran halten. Sie ergänzt die `Befehlsreferenz_Python_PC.md`, die den Weg PC → Test-PCB beschreibt.

Ziele:

- Ein Datenblock beschreibt sich selbst. Python kann die Rohwerte ohne Zusatzwissen in Strom und Zeit umrechnen.
- Die Rohdaten gehen binär über die Leitung, damit die Übertragung kurz bleibt.
- Übertragungsfehler werden über eine Prüfsumme erkannt.

2 Aufbau eines Datenblocks

Ein Datenblock besteht aus drei Teilen:

```
#HDR TYPE=MSM_TRANSIENT N=957 RANGE=0 AVG=100 DT_US=0
     DT_MIN_US=1000 LOG_Q24=23200 T_START_US=1000
     T_STOP_US=1000000 CYCLE=2 T_STRESS_MS=11000\r\n
<N × 2 Byte Rohdaten, uint16, little endian>
#END SUM=48213\r\n
```

(Die Headerzeile ist hier nur zur Lesbarkeit umgebrochen. Gesendet wird sie als **eine** Zeile.)

1. **Headerzeile (ASCII):** beginnt mit `#HDR`, danach Felder `NAME=wert` mit je einem Leerzeichen dazwischen, Abschluss `\r\n`. Alle Werte sind ganze Zahlen ohne Vorzeichen, nur `TYPE` ist Text.
2. **Rohdaten (binär):** genau `N` Werte zu je 2 Byte, zuerst das niederwertige Byte (little endian). Jeder Wert ist der **Mittelwert aus `AVG` ADC-Wandlungen**, als 16-Bit-Rohwert.
3. **Endzeile (ASCII):** `#END SUM=<Prüfsumme>\r\n`.

Bei MSM kommt pro Messung ein eigener Datenblock. Zeilen, die nicht mit `#HDR` beginnen (z. B. `FETCH READY`), ignoriert Python beim Warten auf einen Block.

3 Header-Felder

Feld	Einheit	Bedeutung
TYPE	Text	`IDVG`, `TRANSIENT`, `SPOT`, `MSM_IDVG` oder `MSM_TRANSIENT`

Feld	Einheit	Bedeutung
N	–	Anzahl Messpunkte in diesem Block (tatsächlich gemessen, nach ABORT evtl. weniger)
RANGE	–	Strombereich während der Messung: 0 = 100 nA (Rf 10 MΩ), 1 = 10 µA (Rf 100 kΩ)
AVG	–	Wandlungen, die pro Punkt gemittelt wurden
DT_US	µs	fester Abstand zwischen zwei Punkten (lineare Zeitachse), sonst 0
DT_MIN_US	µs	kleinster möglicher Abstand = Dauer eines Messpunkts (logarithmische Zeitachse), sonst 0
LOG_Q24	–	Wachstumsfaktor der logarithmischen Zeitachse als Festkommazahl (Kapitel 5.2), sonst 0
T_START_US	µs	Zeitpunkt des ersten Punkts, gemessen ab „Stress aus“ bzw. ab Messstart
T_STOP_US	µs	Ende des Messfensters (logarithmische Zeitachse), sonst 0
CYCLE	–	MSM-Zyklus: 0 = Referenzmessung vor dem ersten Stress, dann 1, 2, …; sonst 0
T_STRESS_MS	ms	**gesamte** bisherige Stresszeit (Summe aller Zyklen) bei MSM; sonst 0

Nicht verwendete Felder werden **immer mitgeschickt und auf 0 gesetzt**. So hat der Header immer dieselbe Form.

4 Welche Felder bei welcher Messart

TYPE	N	Zeitachse	benutzte Zeitfelder
IDVG	Sweep-Punkte	keine, x-Achse = V_G	–
TRANSIENT	Punkte	linear	T_START_US, DT_US
SPOT	1	t = T_START_US	T_START_US, CYCLE, T_STRESS_MS
MSM_IDVG	Sweep-Punkte	keine, x-Achse = V_G	CYCLE, T_STRESS_MS
MSM_TRANSIENT	≤ MSM_MAX_POINTS	logarithmisch	T_START_US, T_STOP_US, DT_MIN_US, LOG_Q24, CYCLE, T_STRESS_MS

Bei SPOT ist T_START_US = MSM_MEAS_DELAY in µs.

x-Achse bei ID-VG: Python kennt `Sweep_Start` und `Sweep_Step`, weil es sie selbst geschickt hat: $V_{G,i}$ = Start + i · Step.

5 Zeitachse

5.1 Linear (TRANSIENT)

t_i = `T_START_US` + i · `DT_US`, mit i = 0 … N−1.

`DT_US` ist der **tatsächlich eingestellte** Abstand. Der Timer kann nicht jede Frequenz exakt treffen; die Firmware rechnet ihn aus Prescaler und Reload-Wert zurück.

5.2 Logarithmisch mit Mindestabstand (MSM_TRANSIENT)

Die Recovery läuft ungefähr logarithmisch in der Zeit ab. Deshalb liegen die Punkte logarithmisch verteilt. Am Anfang wäre der log-Abstand aber kleiner, als ein Messpunkt dauert (`DT_MIN_US` = `AVG` × Zeit pro Wandlung). Dort wird deshalb so schnell wie möglich gemessen, also im Abstand `DT_MIN_US`. Sobald der log-Abstand größer ist, geht es logarithmisch weiter.

Die Zeitpunkte werden **nur mit ganzen Zahlen** berechnet. Firmware und Python kommen damit auf exakt dieselben Werte, ohne Rundungsunterschiede zwischen float auf dem STM32 und double in Python:

```
t_0     = T_START_US
gap     = max( DT_MIN_US , (t_i * LOG_Q24) >> 24 )
t_(i+1) = t_i + gap
```

Die Firmware hört auf, wenn t > `T_STOP_US` wird oder `MSM_MAX_POINTS` erreicht ist. Die tatsächliche Anzahl steht in `N`.

Woher kommt `LOG_Q24`? Die Firmware rechnet einmal vor der Messung den idealen Faktor zwischen zwei log-Punkten aus:

r = (T_{stop} / T_{start})^1/(N_{max}−1)

und speichert r − 1 als Festkommazahl mit 24 Nachkommabits:

`LOG_Q24` = round((r − 1) · 2^{24})

`(t * LOG_Q24) >> 24` ist dann t · (r − 1), also der log-Abstand.

Beispiel: T_{start} = 1 ms, T_{stop} = 1 s, N_{max} = 5000, `DT_MIN_US` = 1000:

- r = $1000^{1/4999}$ ≈ 1,001383, also `LOG_Q24` ≈ 23 200 (je nach float-Rundung ±1; egal, weil der Wert im Header steht)
- am Anfang: log-Abstand 1,4 µs < 1000 µs, deshalb linear im 1-ms-Raster
- ab t ≈ 0,72 s: log-Abstand > 1 ms, ab da logarithmisch
- insgesamt ergeben sich etwa **960 Punkte**, nicht 5000

Merke: In einer kurzen Recovery passen höchstens ≈ T_{stop} / `DT_MIN_US` Punkte. Mehr Punkte am Anfang gibt es nur mit kleinerem `AVG`, also schnellerer, aber verrauschterer Messung.

6 Prüfsumme

`SUM` = Summe aller N Rohwerte (als uint16), modulo 65536.

Python rechnet die Summe nach. Stimmt sie nicht, ist der Block fehlerhaft und wird verworfen bzw. die Messung wiederholt. Eine CRC16 wäre sicherer, die einfache Summe reicht aber für einzelne gekippte Bits und verlorene Bytes.

7 Ablauf bei MSM

Zeitpunkt	Block
vor dem ersten Stress (`MSM_INITIAL_MEASURE= TRUE`)	`CYCLE=0 T_STRESS_MS=0`
nach Stress 1 (z. B. 1 s)	`CYCLE=1 T_STRESS_MS=1000`
nach Stress 2 (10 s)	`CYCLE=2 T_STRESS_MS=11000`
nach Stress 3 (100 s)	`CYCLE=3 T_STRESS_MS=111000`
…	…

Gesendet wird jeder Block **während des nächsten Stresses**, der letzte nach Ende der Messung. Danach schickt die Firmware `#DONE\r\n`, damit Python weiß, dass nichts mehr kommt.

Wichtig für die Firmware: Die Stresszeit muss über den Timer gemessen werden (Startzeitpunkt merken, dann bis Ende warten). Ein `wait_ms(stresszeit)` *nach* dem Senden wäre falsch, weil sich dadurch der Stress um die Sendedauer verlängern würde.

8 Vorschlag Firmware: `send_measurement_data()`

Neue Dateien `DATA_SEND.h` / `DATA_SEND.c`. Sie verwenden nur die vorhandenen Funktionen `uart_put_char()` und `uart_put_string()` aus `UART.c`.

8.1 DATA_SEND.h

```
#ifndef DATA_SEND_H
#define DATA_SEND_H

#include <stdint.h>

typedef enum {
    DATA_IDVG,
    DATA_TRANSIENT,
    DATA_SPOT,
    DATA_MSM_IDVG,
    DATA_MSM_TRANSIENT
} Data_TypeTyp;

typedef struct {
    Data_TypeTyp type;
    uint32_t n;            // Anzahl Punkte
    uint32_t range;        // 0 = 100 nA, 1 = 10 uA
    uint32_t avg;          // Mittelungen pro Punkt
    uint32_t dt_us;        // linearer Abstand [us], sonst 0
    uint32_t dt_min_us;    // Mindestabstand log [us], sonst 0
    uint32_t log_q24;      // (r-1) * 2^24, sonst 0
    uint32_t t_start_us;   // erster Punkt [us]
    uint32_t t_stop_us;    // Ende Messfenster [us], sonst 0
    uint32_t cycle;        // MSM-Zyklus, sonst 0
    uint32_t t_stress_ms;  // gesamte Stresszeit [ms], sonst 0
} Data_Header;

void     send_measurement_data(const Data_Header *hdr, const uint16_t *data);
void     send_measurement_done(void);
```

```
uint32_t log_calc_q24(uint32_t t_start_us, uint32_t t_stop_us, uint32_t n_max);
uint32_t log_next_time(uint32_t t_us, uint32_t dt_min_us, uint32_t q24);

#endif
```

8.2 DATA_SEND.c

```
#include <stdint.h>
#include <math.h>
#include "UART.h"
#include "DATA_SEND.h"

static const char *const type_names[] = {
    "IDVG", "TRANSIENT", "SPOT", "MSM_IDVG", "MSM_TRANSIENT"
};

/* vorzeichenlose 32-Bit-Zahl als Text senden
   (uart1_put_int arbeitet mit int und laeuft ab 2^31 ueber) */
static void uart_put_u32(uint32_t v)
{
    char buf[10];
    int i = 0;

    if (v == 0)
    {
        uart_put_char('0');
        return;
    }
    while (v > 0)
    {
        buf[i++] = (char)('0' + (v % 10));
        v /= 10;
    }
    while (i > 0)
    {
        uart_put_char(buf[--i]);
    }
}

static void put_field(const char *name, uint32_t value)
{
    uart_put_char(' ');
    uart_put_string((char*)name);
    uart_put_char('=');
    uart_put_u32(value);
}

void send_measurement_data(const Data_Header *hdr, const uint16_t *data)
{
    uint32_t i;
    uint16_t sum = 0;

    /* ---- Header ---- */
    uart_put_string("#HDR TYPE=");
    uart_put_string((char*)type_names[hdr->type]);
    put_field("N",           hdr->n);
```

```
    put_field("RANGE",       hdr->range);
    put_field("AVG",         hdr->avg);
    put_field("DT_US",       hdr->dt_us);
    put_field("DT_MIN_US",   hdr->dt_min_us);
    put_field("LOG_Q24",     hdr->log_q24);
    put_field("T_START_US",  hdr->t_start_us);
    put_field("T_STOP_US",   hdr->t_stop_us);
    put_field("CYCLE",       hdr->cycle);
    put_field("T_STRESS_MS", hdr->t_stress_ms);
    uart_put_string("\r\n");

    /* ---- Rohdaten binaer, little endian ---- */
    for (i = 0; i < hdr->n; i++)
    {
        uint16_t v = data[i];
        uart_put_char((char)(v & 0xFF));   // Low-Byte zuerst
        uart_put_char((char)(v >> 8));     // High-Byte
        sum += v;                          // Ueberlauf = modulo 65536
    }

    /* ---- Ende + Pruefsumme ---- */
    uart_put_string("#END SUM=");
    uart_put_u32(sum);
    uart_put_string("\r\n");
}

void send_measurement_done(void)
{
    uart_put_string("#DONE\r\n");
}

/* Faktor fuer log. Zeitachse, einmal vor der Messung berechnen */
uint32_t log_calc_q24(uint32_t t_start_us, uint32_t t_stop_us, uint32_t n_max)
{
    float r;

    if (n_max < 2 || t_start_us == 0 || t_stop_us <= t_start_us)
        return 0;

    r = powf((float)t_stop_us / (float)t_start_us, 1.0f / (float)(n_max - 1));
    return (uint32_t)((r - 1.0f) * 16777216.0f + 0.5f);   // * 2^24
}

/* naechster Messzeitpunkt [us] - nur Ganzzahl-Rechnung,
   damit Python exakt dieselben Zeiten erhaelt */
uint32_t log_next_time(uint32_t t_us, uint32_t dt_min_us, uint32_t q24)
{
    uint32_t gap = (uint32_t)(((uint64_t)t_us * q24) >> 24);

    if (gap < dt_min_us)
        gap = dt_min_us;

    return t_us + gap;
}
```

8.3 Verwendung (Beispiel MSM-TRANSIENT, ein Zyklus)

```
uint16_t    buffer[5000];          // 10 kB, global/static anlegen, nicht auf dem Stack
Data_Header hdr;
uint32_t    t, n = 0;
uint32_t    q24 = log_calc_q24(t_start, t_stop, param->msm_max_points);

t = t_start;
while (t <= t_stop && n < param->msm_max_points && !abort_requested)
{
    /* warten bis Zeit t (ab "Stress aus") erreicht ist, dann AVG Wandlungen mitteln */
    buffer[n++] = /* gemittelter Rohwert */ 0;
    t = log_next_time(t, dt_min, q24);
}

hdr.type        = DATA_MSM_TRANSIENT;
hdr.n           = n;
hdr.range       = param->sense_range;
hdr.avg         = param->sense_average;
hdr.dt_us       = 0;
hdr.dt_min_us   = dt_min;
hdr.log_q24     = q24;
hdr.t_start_us  = t_start;
hdr.t_stop_us   = t_stop;
hdr.cycle       = zyklus;
hdr.t_stress_ms = stress_summe_ms;

/* naechsten Stress einschalten + Startzeit merken, DANN senden */
send_measurement_data(&hdr, buffer);
```

Hinweise:

- `send_measurement_data()` **blockiert**, bis alles gesendet ist (≈ 0,87 s bei 5000 Punkten und 115200 Baud, ≈ 10,4 s bei 9600 Baud). Das ist in Ordnung, solange es während des Stresses passiert und die Stresszeit über den Timer gemessen wird. Später kann man auf UART-TX per DMA umstellen; das Datenformat bleibt dann gleich.
- ABORT funktioniert auch während des Sendens, weil er im RX-Interrupt erkannt wird.
- `powf` braucht `math.h`. Der Cortex-M3 hat keine FPU, die Berechnung läuft in Software, aber nur **einmal** pro Messung.
- Der Buffer mit 10 kB muss `static` bzw. global sein. Der Stack ist beim STM32-Startup meist nur 1 kB groß.

9 Vorschlag Python: Block empfangen

```
import struct

def zeitachse(hdr):
    n = hdr["N"]
    typ = hdr["TYPE"]
    if typ == "TRANSIENT":
        return [hdr["T_START_US"] + i * hdr["DT_US"] for i in range(n)]
    if typ == "MSM_TRANSIENT":
        t = [hdr["T_START_US"]]
        while len(t) < n:
            gap = max(hdr["DT_MIN_US"], (t[-1] * hdr["LOG_Q24"]) >> 24)
            t.append(t[-1] + gap)
        return t
```

```
    if typ == "SPOT":
        return [hdr["T_START_US"]]
    return None            # IDVG / MSM_IDVG: x-Achse = V_G

def lese_block(ser):
    """Liest einen Datenblock. Gibt (header, rohwerte) zurueck oder None bei #DONE."""
    while True:
        zeile = ser.readline().decode("ascii", errors="ignore").strip()
        if zeile == "":
            raise TimeoutError("keine Daten")
        if zeile == "#DONE":
            return None
        if zeile.startswith("#HDR"):
            break

    hdr = {}
    for teil in zeile.split()[1:]:
        name, wert = teil.split("=", 1)
        hdr[name] = wert if name == "TYPE" else int(wert)

    n = hdr["N"]
    roh = ser.read(2 * n)                  # genau 2*N Byte, '\n' in den Daten stoert nicht
    if len(roh) != 2 * n:
        raise IOError(f"nur {len(roh)} von {2*n} Byte empfangen")
    werte = list(struct.unpack(f"<{n}H", roh))

    ende = ser.readline().decode("ascii", errors="ignore").strip()
    if not ende.startswith("#END SUM="):
        raise IOError("Endzeile fehlt")
    if int(ende.split("=")[1]) != (sum(werte) & 0xFFFF):
        raise IOError("Pruefsumme falsch")

    hdr["t_us"] = zeitachse(hdr)
    return hdr, werte

# Beispiel MSM: alle Bloecke einsammeln
# bloecke = []
# while (b := lese_block(ser)) is not None:
#     bloecke.append(b)
```

Hinweise:

- **Timeout:** `ser.read(2*n)` wartet höchstens `ser.timeout` Sekunden. Der Timeout muss länger als die Übertragung sein: mindestens 2·N·10 / Baudrate + Reserve. Bei 9600 Baud und 5000 Punkten sind das über 10 s. Bei MSM muss `readline()` außerdem bis zum Ende des nächsten Stresses warten können. Dort `timeout=None` verwenden oder in einer Schleife warten.
- **Rohwert → Strom:** Die Umrechnung hängt von ADC-Referenz, Nachverstärker und Rf (`RANGE`) ab und wird noch in einer eigenen Funktion `raw_to_current(raw, range)` festgelegt. Formel: siehe Firmware-Doku bzw. Schaltplan.

10 Offene Punkte

- `DT_MIN_US` muss **gemessen** werden (Oszilloskop am ADC-CS oder Timer). Er hängt von SPI-Takt, Systemtakt (8 MHz HSI oder 72 MHz PLL) und `AVG` ab.
- Die Funktion `raw_to_current()` in Python ist noch festzulegen.
- Baudrate auf 115200 oder mehr erhöhen. Die Grenze setzen USB-UART-Chip und Optokoppler.

- Optional: CRC16 statt einfacher Summe.

11 Quellen

- Eigener Code (`Software/firmware/PCB_test_code/`): `UART.c`, `main.c`, `SPI.c`, `COMMAND_DECODE.h`
- Eigene Doku: `Befehlsreferenz_Python_PC.md`
- Eigene Doku: `Dokumentation/Firmware_Doku_PCB_test_code.pdf`
- pySerial API (`read`, `readline`, `timeout`): https://pyserial.readthedocs.io/en/latest/pyserial_api.html
- Python `struct` (Format <H = uint16 little endian): https://docs.python.org/3/library/struct.html
- STMicroelectronics RM0008: USART, SPI (BR-Teiler), Timer, RCC (HSI 8 MHz). Liegt in `Software/firmware/Datenblätter_Cortex/`
- ARM Cortex-M3 (keine FPU, 64-Bit-Multiplikation UMULL): PM0056 Programming Manual, ebenfalls in `Datenblätter_Cortex/`
- TI ADS8866 Datenblatt (SBAS606), max. 100 kSPS: https://www.ti.com/product/ADS8866
- B. Kaczer et al., „Ubiquitous Relaxation in BTI Stressing – New Evaluation and Insights“, IEEE IRPS 2008 (logarithmische Zeitachse der Recovery)